
COMP2511

Tutorial 3

Last Week's Tutorial

- Inheritance
- Method Overriding
- Abstract Classes and Interfaces
- Polymorphism

This Week's Tutorial

- Testing
- Debugging
- Exceptions
- Domain Modelling and UML Diagrams

Notices

- Assignment-i has been released on GitLab, and is due Week 5 Wednesday.
 - You should be thinking about the design (what classes will you make, what will each of them do?) before doing any actual coding.
 - You must push to the main branch of your GitLab repository at least twice, with at least 24 hours between them. **There is a max mark of 50% if this requirement is not met.**

Testing - JUnit

- JUnit is the main testing framework we use in this course, similar to what Jest is to COMP1531.
- Each testing suite is separated into its own class, and each of the methods defined in that class will be the individual unit tests.
 - For example, if you were making a test suite to test Task 1 of Assignment 1, this could be put in a file called **Task1Tests.java**, and a method in the class could be **testCreateTrain()**.
- Unit tests are identified using the **@Test** tag over method names, like the **@Override** tag for method overriding.
 - This tag is required to identify a block of code as a JUnit test.
- In each unit test, conditions are checked using **assertions**, such as **assertEquals()**, **assertTrue()** and **assertThrows()**. If any assertion in a unit test fails, the whole test fails.

Testing - JUnit Example (from Assignment 1)

```
21 public class TaskAExampleTests {
22     @Test
23     public void testCreateStations() {
24         TrainsController controller = new TrainsController();
25
26         controller.createStation("station1", "DepotStation", 1.0, 1.0);
27         assertEquals(new StationInfoResponse("station1", "DepotStation", new Position(1.0, 1.0)),
28             controller.getStationInfo("station1"));
29
30         controller.createStation("station2", "CargoStation", 10.0, 10.0);
31         assertEquals(new StationInfoResponse("station2", "CargoStation", new Position(10.0, 10.0)),
32             controller.getStationInfo("station2"));
33
34         controller.createStation("station3", "PassengerStation", 19.6, 15);
35         assertEquals(new StationInfoResponse("station3", "PassengerStation", new Position(19.6, 15)),
36             controller.getStationInfo("station3"));
37
38         assertListAreEqualIgnoringOrder(List.of("station1", "station2", "station3"), controller.listStationIds());
39     }
40 }
```

Testing - IDE Debugging

- We have access to debugging tools that allow us to step through the execution of a program, line-by-line.
- A **breakpoint** is a specific line of code where you would like to pause execution intentionally.
 - In VS Code, they can be set on the area to the left of each of the line numbers (called the *glyph margin*).
 - By running your code in **Debug Mode**, you can pause at breakpoints, allowing you to inspect the state of the program at a given time and analyse the control flow (if statements, while loops...).

Exceptions

- An **exception** is an event that occurs when an error is encountered during the execution of a program.
 - Think of an exception as a **flag** that something has gone wrong, such as dividing by zero or trying to dereference a null object.
 - When the error occurs, an exception object containing information about the error is created and thrown to the Java runtime system to handle.
- In Java, exceptions are classified into two categories:
 - **Checked** exceptions - methods that can throw these exceptions must explicitly declare so in the method signature, and handled in a *try-catch* block (eg. **IOException**).
 - **Unchecked** exceptions - these do *not* need to be included in a method's signature or caught in a try-catch block (eg. **ArithmeticException**, **IllegalArgumentException**).

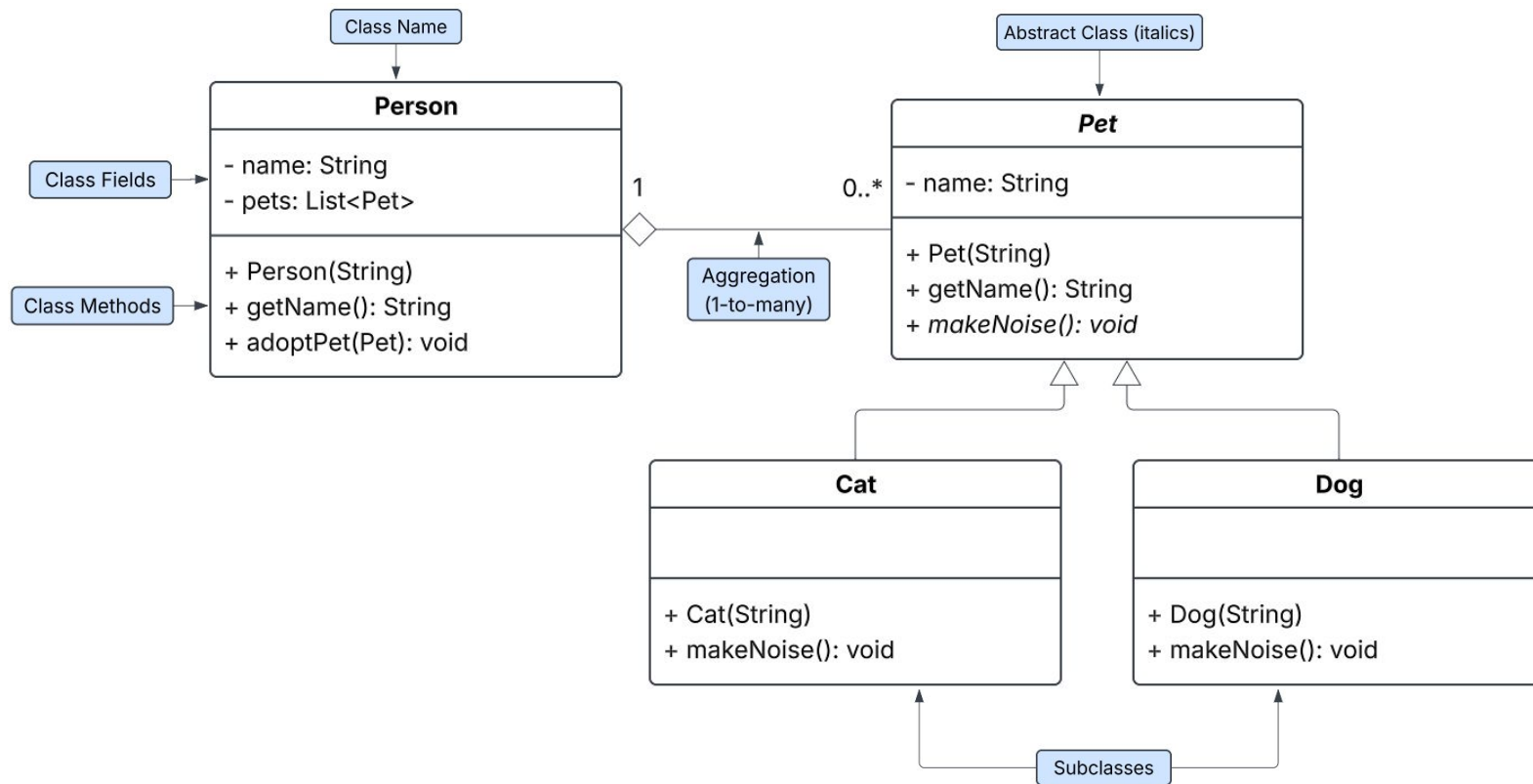
Live Example: Testing, Debugging and Exceptions

- Inside **src/cafe** there is an implementation simulating ordering from a cafe. Write tests for calculating the costs of different orders.
- There is an intentional bug in the code. Use the debug tools to find and fix the bug.
- Afterwards, create a checked exception called **CafeException** that is thrown if an item not on the menu is ordered. Write a corresponding test case for this.

UML Class Diagrams

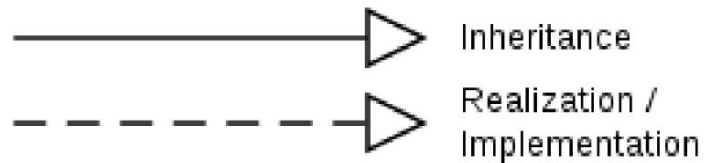
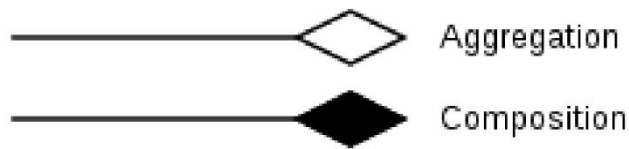
- **UML** (**U**nified **M**odeling **L**anguage) Class Diagrams are used to visually demonstrate the structure of a system in terms of its components and relationships between the components.
- From a UML diagram, we should be able to tell:
 - which classes are present in the system
 - what fields and methods each of the classes have
 - which classes are subclasses of other classes (**is-a** relationships)
 - which classes contain instances of other classes (**has-a** relationships), and the **cardinality** of the relationship (e.g. 1 car has 1 steering wheel, 1 class has many students, many students have 1 teacher)

UML Diagram Components



Relationships in UML Diagrams

- For inheritance/implementation, the arrow points to the parent class being inherited/interface being implemented.
- For 'has-a' relationships, the diamond is on the side of the container.
- There are two types of 'has-a' relationships:
 - **Aggregation:** A contains B. B *can* exist independently of A (i.e. the existence of B is not directly tied to A). For example, a classroom has students.
 - **Composition:** A contains B. B *cannot* exist independently of A (i.e. the existence of B is directly tied to A). For example, a person has thoughts.



Live Example: Domain Modelling

- A Car has one or more engines and a producer.
 - Thermal Engines, which have a default max speed of 114, although they can be produced with a different max speed, and the max speed can change to any value between 100 and 250.
 - Electrical Engines, which have a default max speed of 180. This is the speed at which they are produced, and the max speed can change to any value that is divisible by 6.
- Cars are able to drive to a particular location x, y.

Additionally, we want to model the behaviour of a time travelling car, as well as time travelling for any vehicle. A 'time travelling vehicle' stays in the same location but travels to a LocalDateTime.

Create a UML diagram which models the domain.